Experiment No -08 (Group C)

Title:-

Implementation of Hash Table using Division Method and Chaining

Problem statement:-

To implement a hash table of fixed size (10) using the **division method** as the hash function and **chaining** to handle collisions. The operations to be implemented are:

1. **Insert(key, value)** – To insert a key-value pair into the hash table.
2. **Search(key)** – To find the value associated with a given key.
3. **Delete(key)** – To remove a key-value pair from the hash table.

Pre-requisite:-

- Basic knowledge of **Python programming**
- Understanding of **lists and functions**
- Basic knowledge of **hashing and hash tables**
- Understanding of the **modulo (%) operator**
- Basic knowledge of **key-value pairs**

Objective:-

To understand how a hash table works by storing, searching, and deleting key-value pairs, while handling collisions using chaining and calculating positions using the division method.

Software and Hardware requirements:-

1. **Operating system:** Linux- Ubuntu 16.04 to 17.10, or Windows 7 to 10,
2. **RAM-** 2GB RAM (4GB preferable)
3. IDE :- Python 3

Outcome:-

After completing this practical, students will be able to:

- Understand the basic concept of a **hash table**.
- Apply the **division method** for calculating the hash index.
- Understand and handle **collisions using chaining**.
- Implement **insert, search, and delete** operations.
- Develop a basic understanding of the efficiency of hashing.

Theory:-

Introduction to Hashing

Hashing is the technique of mapping a large chunk of data into small tables using a hash function.

As we know when we try to search any data item like linear search or binary search time required for searching is dependent on total number of elements stored in that data structure.

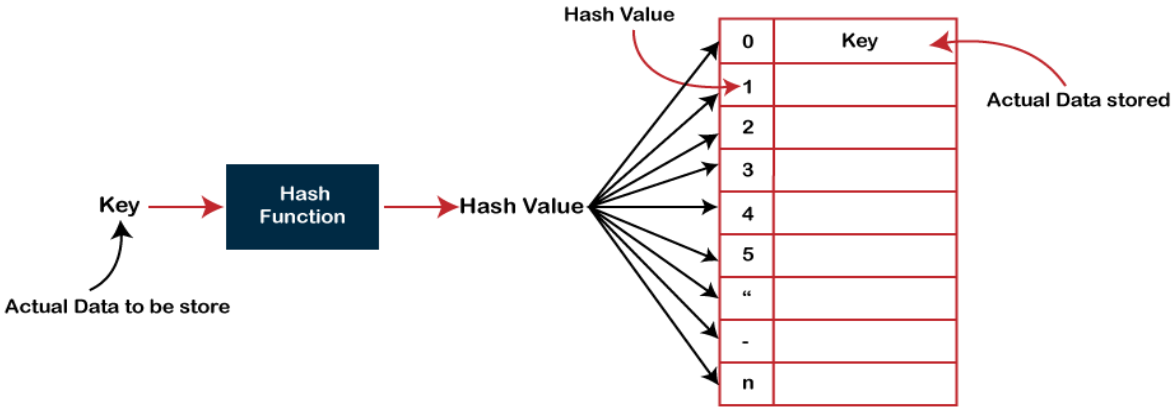
Hash Table

A hash table is a data structure that stores records in an array, called a hash table.

It is data structure where data elements are stored based on their hash key values generated by hash function.

On these stored data elements operation like insertion, deletion and search can be performed with constant time complexity. This Hash key value is generated for any data elements by using hash function.

In short we use Hash function to generate key values and using these key value data elements is stored at particular position.



Advantages of Hash table

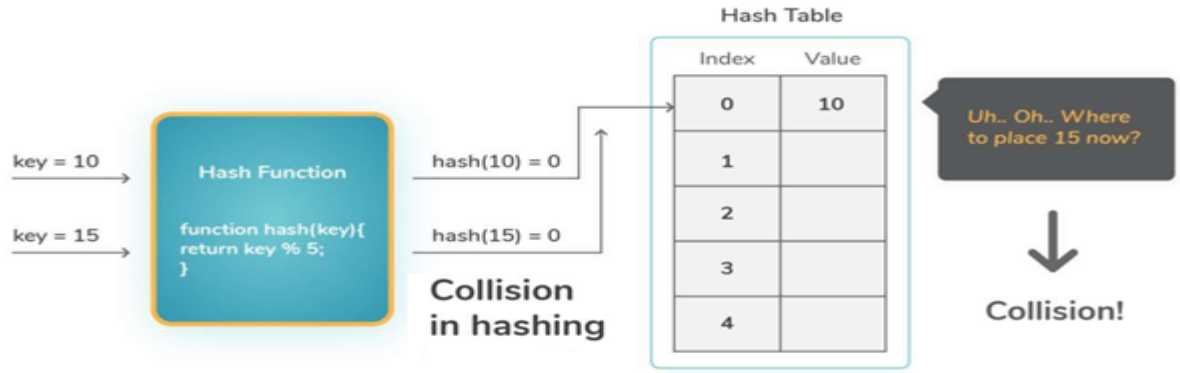
1. Hash tables have high performance when looking up data, inserting, and deleting existing values.
2. The time complexity for hash tables is constant regardless of the number of items in the table.
3. They perform very well even when working with large datasets.

Disadvantages of Hash table

1. You cannot use a null value as a key.
2. Collisions cannot be avoided when generating keys using hash functions. Collisions occur when a key that is already in use is generated.
3. If the hashing function has many collisions, this can lead to performance decrease.

Collision

A collision occurs when more than one values are mapped to same index in the hash table by particular hash function.



Division Method

This is the most simple and easiest method to generate a hash value for any data item k which you want top store in the hash table.

The hash function divides the value k by M and then uses the remainder obtained.

h(K) = k mod M Here, k is the key value, and M is the size of the hash table.

It is best suited that M is a prime number as that can make sure that keys are more uniformly distributed.

Example:

k = 12345 , M = 95

h(12345) = 12345 mod 95= 90

k = 1276, M = 11

h(1276) = 1276 mod 11 = 0

Advantage of Division Method-

1. This method is good for any value of M.
2. It is fast as it requires only single division operation.

Disadvantage of Division Method-

1. This method leads to poor performance since consecutive keys map to consecutive indices in the hash table.

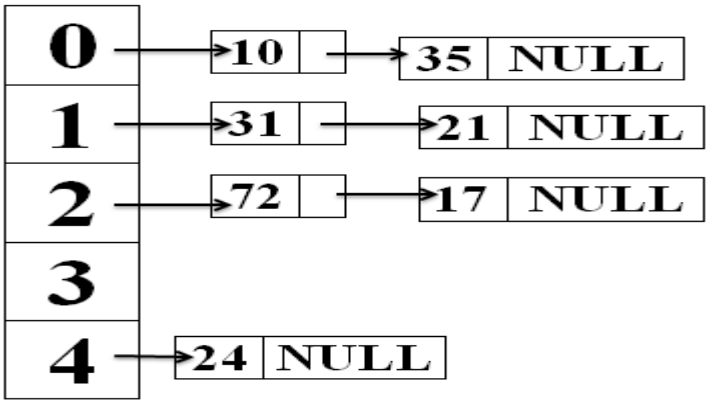
Open Hashing / Separate Chaining

In Open Hashing, one of the methods used to resolve the collision is known as a Sseparate chaining method.

In Open Hashing , we maintain a linked list at every hash index for colliding keys. Suppose we wish to store a set of number {10,31,21.72,24,35,17 } into hash table of size 5.

Assume that we have hash function H such that **H(key)=key % 5.** So we will get respective indices for given key values as follows after applying hash function on keys.

H(10)=10 % 5=0
H(31)=31 % 5=1
H(21)=21 % 5=1
H(72)=72 % 5=2
H(24)=24 % 5=4
H(35)=35 % 5=0
H(17)=17 % 5=2



Clearly 10& 35 , 31 & 21 as well as 72 &17 will have same hash value means they are mapped to same index.

In this case we simply append the colliding values to a list pointed by their hash value as shown in diagram.

Advantages-

1. Open hashing method is simple to implement & understand.
2. The hash table never ends, so we can add new elements always.

Disadvantages-

1. The cache performance is poor as the key are stored using singly linked list.
2. The storage space is wasted as some part of hash table is never used.
3. Extra storage is used for storing links of the chain.

Algorithm for Hash Table Using Division Method and Chaining

1: Initialize Hash Table

Step 1: Set the hash table size to 10.

Step 2: Create 10 empty lists (chains).

Step 3: Store these lists in the hash table.

2.Hash Function

Step 1: Take the given key.

Step 2: Calculate the index using the division method: index = key % 10

Step 3: Return the calculated index.

3: Insert(key, value)

Step 1: Take key and value as input.

Step 2: Calculate the index: index = key % 10

Step 3: Go to the chain at that index.

Step 4: Add [key, value] to the chain using append().

4: Search(key)

Step 1: Take the key to be searched.

Step 2: Calculate the index: index = key % 10

Step 3: Go to the chain at that index.

Step 4: Traverse each item in the chain.

Step 5: Compare the key with item[0].

Step 6: If the key matches, return item[1] (the value).

Step 7: If the entire chain is searched and the key is not found, return "Key not found"

5: Delete(key)

Step 1: Take the key to be deleted.

Step 2: Calculate the index: index = key % 10

Step 3: Go to the chain at that index.

Step 4: Traverse each item in the chain.

Step 5: Compare the key with item[0].

Step 6: If the key matches, remove that item from the chain.

Step 7: Return "Key deleted".

Step 8: If the key is not found, return "Key not found".

6: Display

Step 1: Set i = 0.

Step 2: Print the index and its corresponding chain.

Step 3: Increment i.

Step 4: Repeat Steps 3–4 until i = 10..

Conclusion:-

The hash table was successfully implemented using the **division method** with a table size of 10. **Chaining** was used to handle collisions when two or more keys produced the same hash index. The **insert, search, and delete** operations were successfully performed. This practical helps in understanding how hashing provides an efficient way to store and retrieve data.

Viva Questions:-

1. What is a hash table?
2. What is a key?
3. What is a hash function?
4. Why is the modulo **%** operator used?
5. Which hash function is used here?
6. What is the division method?
7. What is a collision?
8. How are collisions handled in your program?
9. What does item[0] represent?
10. What is the difference between a hash table and an array?